

BUỔI THỰC HÀNH: LẬP TRÌNH PYTHON VỚI GIAO THỨC AMQP

1. Mục tiêu buổi thực hành

Sau buổi này, sinh viên cần:

- Hiểu cách một ứng dụng Python kết nối tới AMQP broker.
- Biết cách gửi và nhận message qua **queue**.
- Biết sử dụng các thành phần cơ bản của AMQP như **producer, queue, consumer, exchange, routing key**.
- Vận dụng AMQP để mô phỏng bài toán giám sát và xử lý dữ liệu IoT.

2. Yêu cầu môi trường

Sinh viên chuẩn bị:

- Python 3.x
- Thư viện `pika`
- RabbitMQ đã cài sẵn trên máy hoặc broker dùng chung của lớp
- VS Code / PyCharm / terminal

Cài thư viện:

```
pip install pika
```

Bài 1. Gửi và nhận message cơ bản qua queue

Mục tiêu

Làm quen với mô hình **producer** → **queue** → **consumer** trong AMQP.

Yêu cầu

Viết 2 chương trình Python.

Chương trình 1: Producer

- Kết nối tới RabbitMQ broker.

- Tạo hoặc sử dụng queue có tên:

`iot_lab_queue`

- Gửi một message chứa:
 - Họ tên sinh viên
 - Mã sinh viên
 - Nội dung chào mừng

Ví dụ:

`Xin chào tu ung dung Python AMQP - B21xxxx - Nguyen Van A`

Chương trình 2: Consumer

- Kết nối tới cùng broker.
- Lắng nghe queue:

`iot_lab_queue`

- Khi nhận được message, in ra:
 - Nội dung message
 - Thời điểm nhận

Yêu cầu đầu ra

Ví dụ:

Da nhan message: Xin chào tu ung dung Python AMQP - B21xxxx - Nguyen Van A
Thoi gian nhan: 14:05:21

Gợi ý mở rộng

- Cho phép producer gửi nhiều message liên tiếp.
- Cho consumer chạy liên tục cho tới khi dừng bằng `Ctrl+C`.

Đánh giá

- Kết nối được broker
- Gửi được message vào queue
- Consumer nhận và hiển thị đúng

Bài 2. Mô phỏng cảm biến IoT gửi dữ liệu môi trường

Mục tiêu

Mô phỏng thiết bị IoT gửi telemetry qua AMQP.

Yêu cầu

Viết 2 chương trình Python.

Chương trình 1: Sensor Producer

- Mô phỏng cảm biến gửi dữ liệu mỗi 3 giây.
- Gửi dữ liệu vào queue:

```
sensor_data_queue
```

- Dữ liệu gửi ở dạng JSON, gồm:

```
{  
  "device_id": "sensor01",  
  "temperature": 29.5,  
  "humidity": 62.1  
}
```

- Giá trị nhiệt độ và độ ẩm được sinh ngẫu nhiên.

Chương trình 2: Monitoring Consumer

- Nhận dữ liệu từ queue:

```
sensor_data_queue
```

- Phân tích dữ liệu và in ra:
 - mã thiết bị
 - nhiệt độ
 - độ ẩm
- Nếu:
 - nhiệt độ > 35 thì cảnh báo "CANH BAO: Nhiệt độ cao"
 - độ ẩm < 40 thì cảnh báo "CANH BAO: Độ ẩm thấp"

Yêu cầu đầu ra

Ví dụ:

```
Device: sensor01
Temperature: 36.4
Humidity: 38.9
CANH BAO: Nhiệt độ cao
CANH BAO: Độ ẩm thấp
```

Gợi ý mở rộng

- Bổ sung trường thời gian gửi.
- Mô phỏng 2 hoặc 3 cảm biến khác nhau.

Đánh giá

- Payload đúng định dạng JSON
- Gửi dữ liệu tuần hoàn
- Consumer xử lý đúng và phát hiện cảnh báo

Bài 3. Mô phỏng hệ thống điều phối cảnh báo IoT với exchange

Mục tiêu

Làm quen với **exchange** và **routing key** trong AMQP.

Yêu cầu

Viết 3 chương trình Python.

Chương trình 1: Alert Producer

- Tạo một `direct exchange` tên:

```
iot_alert_exchange
```

- Gửi các cảnh báo với các mức:
 - `info`
 - `warning`
 - `critical`

Ví dụ message:

```
Nhiệt độ phòng máy chuột mức warning
```

Routing key tương ứng:

- `info`
- `warning`
- `critical`

Chương trình 2: Warning Consumer

- Tạo queue:

```
warning_queue
```

- Bind với exchange `iot_alert_exchange` bằng routing key:

```
warning
```

- Chỉ nhận các cảnh báo mức `warning`.

Chương trình 3: Critical Consumer

- Tạo queue:

```
critical_queue
```

- Bind với exchange `iot_alert_exchange` bằng routing key:

```
critical
```

- Chỉ nhận các cảnh báo mức `critical`.

Yêu cầu đầu ra

Ví dụ khi producer gửi 3 message:

- `info`: không consumer nào ở trên nhận
- `warning`: chỉ `warning_queue` nhận
- `critical`: chỉ `critical_queue` nhận

Ví dụ:

```
[warning_queue] Da nhan: Nhiệt độ phòng máy vượt ngưỡng warning  
[critical_queue] Da nhan: Camera kho lạnh mất kết nối critical
```

Gợi ý mở rộng

- Thêm một consumer nhận tất cả cảnh báo bằng `topic exchange`.

- Ghi log cảnh báo ra file.

Đánh giá

- Tạo được exchange
- Bind queue đúng routing key
- Message được phân phối đúng nơi nhận

3. Yêu cầu nộp bài

Mỗi sinh viên hoặc nhóm nộp:

- `producer_bai1.py`
- `consumer_bai1.py`
- `sensor_producer_bai2.py`
- `monitor_consumer_bai2.py`
- `alert_producer_bai3.py`
- `warning_consumer_bai3.py`
- `critical_consumer_bai3.py`

Kèm file `README.txt` mô tả:

- Broker sử dụng
- Cách chạy từng chương trình
- Kết quả đạt được

Deadline nộp bài: 23:59 thứ Tư 15/04/2026

Nộp bài qua github cá nhân, gửi link repo chứa bài làm vào nhóm zalo của lớp